Functional Programming

⇒ Functional programming is a programming paradigm in which computation is treated as evaluation of mathematical function.

⇒ Lambda function is a anonymous function i.e: function without name.

⇒ Lambda is a keywood which is used to crack anonymous fn

- def fun(n):
    return n**2
  print(fun(4))

O/p ⇒ 16

* Lambda n: n**2

- a = lambda n: n**2
  print(a(4))

O/p ⇒ 16

print(type(a)

O/p ⇒ <class/function>

```
x = lambda a,b : a+b
print(x(3,4))
```

O/p ⇒ 7

⇒ Generally we use lambda function, when the function is of single line or the function to be used only once.

- lambda x,y : x+y
- lambda x,y : x-y
- lambda a,b,x,y = a+b*x-y

⇒ Lambda functions can have any no. of input arguments.

```
lf1 = lambda      :10
lf2 = lambda      :print(10)
lf1() ⇒ return 10
lf2() ⇒ print 10 and return None.
```

- a = lambda (7) x : lambda y : x+y
  c = a(7) → lambda y = 7+y
  print (c(9))

  ⇒ 16

- def fun(n): (n=2)
    return lambda a : a*n
  ~~b = fun(2)~~
  l = fun(2) ⇒ lambda a : a*2
  print(l(7)) ⇒ 14

  a=7

⇒ lambda functions
⇒ filter, map, reduce
⇒ problem solving.

Functional

✱ filter ()

⇒ syntax : filter (function, iterable)

- lst = [10, 45, 72, 91, 63, 48, 51]
  def fun(n):
    if n>50:
      return True.
    else
      return False.
  a = list (filter (fun, lst))
  print(a)

  O/p ⇒ [72, 91, 63, 51]

- lst = [1, 7, 11, 12, 9, 3, 13, 21]
  a = list (filter (lambda x : x>10, lst))
  print(a)

  O/p ⇒ [11, 12, 13, 21]

- fruits = ["apple", "banana", "orange"]
  l1 = list (filter (lambda x : len(x)>5, fruits))
  print(l1)

  O/p ⇒ ["banana", "orange"]

Que data = [5,12,17,18,24,32]
filtered = list(filter(lambda x : x%2==0, data))
print(filtered)

A) [12,18,24,32]
B) [5,17]
C) [5,12,17]
D) [18,24,32]

Que def apply(f,x):
    return f(x)
print(apply(lambda a: a**3, 3))
f

A) 3
B) 27
C) 81
D) Error

⇒ 3**3
⇒ 27

★ map()

⇒ syntax : map(function, iterable)

- a = list(map(lambda x : x**2, [1,2,3,4]))
  print(a)
  ⇒ O/p ⇒ [1,4,9,16]

- def fun(n):
    return n+5
  ⇒ O/p [6,7,8,9]

* x = list(map(fun, [1,2,3,4]))
  print(x)

- l1 = ["hello","world"]
  y = list(map(str.upper, l1))
  print(y) ⇒ ["HELLO","WORLD"]
  O/p

- l1 = [1,2,3]
  l2 = [4,5,6]

  ∴ O/P ⇒ [5,7,9]

  a = list(map(lambda x,y : x+y, l1, l2))

★ reduce()

⇒ syntax ⇒ reduce(function, iterable, initializer)

It always take 2 inputs (i.e. no. of inputs: 2)

initializer ↓ optional

- a = reduce(lambda x,y : x+y, [1,2,3,4])

  print(a)

  ⇒ 10

  Step1 : 1+2 = 3

  Step2 = 3+3 = 6

  Step3 = 6+4 = 10

- a = reduce(lambda x,y : f(x,y), [a,b,c,d])

  Step1 ⇒ res1 = f(a,b)

  Step2 ⇒ res2 = f(res1, c)

  Step3 ⇒ res3 = f(res2, d)

- a = reduce(lambda x,y : f(x,y), [a,b,c,d], 1)

  Step1 ⇒ res1 = f(1,a)

  Step2 ⇒ res2 = f(res1, b)

  Step3 ⇒ res3 = f(res2, c)

  Step4 ⇒ res4 = f(res3, d)

Initializer:

⇒ If given, start from this and take one value from iterable.

⇒ If not given, start from iterable by taking first two values.

Qns

- c = reduce(lambda x,y : x*y, range(1,6))

  print(c)

  ⇒ 120

- d = reduce(lambda x,y : x+y, [1,2,3,4], 5)

  print(d)

  ⇒ 15

  5+1+2+3+4

- e = reduce (lambda x,y : x if x>y else y, [7,1,10,2,19,11])
  print(e)

  7,1 ⇒ 7
  7,10 ⇒ 10
  10,2 ⇒ 10
  10,19 ⇒ 19
  19,11 ⇒ 19

  ⇒ **19**

- a = reduce (lambda x,y : x+y if y%2==0 else x, [1,2,3,4,5],0)
  print(a)

  0,1 ⇒ 0
  0,2 ⇒ 2
  2,3 ⇒ 2
  2,4 ⇒ 6
  6,5 ⇒ 6

  ⇒ **6**

- b = reduce (lambda x,y : x + "_" + y, ["RAVI", "ISHAN", "ADITYA"])
  print(b)

  ⇒ **RAVI_ISHAN_ADITYA**

- a = reduce (lambda x,y : x-y, [1,2,3,4])
  print(a)

  +1 - 2 = -1
  -1 - 3 = -4
  -4 - 4 = -8

  ⇒ **-8**

- d = list (map (lambda x : x[0] * x[1], [(2,3), (5,6)]))
  print(d)

  x = (2,3)
  x[0] = 2
  x[1] = 3
  x = 6

  x = (5,6)
  x[0] = 5
  x[1] = 6
  x = 30

  ⇒ **[6,30]**